

Experiment 1

Study of Software Types and Attributes of Good Software

1. Objective

To identify various software types and analyze key attributes constituting good software by evaluating real-world systems based on usability, reliability, maintainability, and efficiency.

2. Theory

2.1 Software Types

System Software: The platform for other software, managing hardware (e.g., Windows, Linux).

Application Software: End-user software for specific tasks (e.g., MS Office, Google Chrome).

Programming Software: Developer tools for creating software (e.g., compilers, IDEs).

Embedded Software: Controls non-computer hardware (e.g., microwaves, digital watches).

2.2 Attributes of Good Software

Usability: Ease of learning and operating. Intuitive and straightforward.

Reliability: Consistency in performing intended functions without failure.

Maintainability: Ease of modification, fixing bugs, and improving performance.

Efficiency: Optimal use of system resources (CPU, memory) during tasks.

3. Procedure

Select two real-world systems: Microsoft Word and Facebook.

Identify their software types.

Evaluate each against usability, reliability, maintainability, and efficiency.

Record findings in a comparative table.

Draw conclusions based on the analysis.

4. Practical Work

Attribute	Microsoft Word	Facebook
Software Type	Application Software (Word Processor)	Application Software (Social Media Web App)
Usability	High. Features the Ribbon UI and a WYSIWYG editor.	High. Core features (posting, liking, scrolling) are highly intuitive.

Attribute	Microsoft Word	Facebook
Reliability	High. Auto-save prevents data loss, ensuring dependability for work.	Moderate/High. High core uptime, though small integrations may experience bugs.
Maintainability	High. Microsoft provides consistent patches and updates.	High. Microservices architecture allows independent component updates.
Efficiency	Moderate/High. Runs smoothly, but degrades with massive, complex files.	High. Optimized backend handles millions of concurrent users efficiently.

5. Conclusion

Both MS Word and Facebook successfully meet the criteria of good application software through balanced usability, reliability, maintainability, and efficiency. Attribute importance shifts by domain: Word prioritizes reliability for data retention, while Facebook emphasizes usability and efficiency for mass engagement.

Experiment 2

Case Study on Software Process Models

1. Objective

To compare software process models by applying Waterfall, Incremental, and Prototyping to a Credit Card Fraud Detection System.

2. Theory

Waterfall Model: Linear progression (Requirements, Design, Implementation, Testing, Deployment). Inflexible; suited for stable requirements.

Incremental Model: Built in successive increments starting with core features, integrating early feedback.

Prototyping Model: Early prototype built for user feedback to refine unclear requirements before final development.

3. Procedure

Select "Credit Card Fraud Detection System" for modeling.

Conceptualize development using Waterfall, Incremental, and Prototyping.

Describe stages for each model's application.

Create tabular/visual representations.

Perform a comparative suitability analysis.

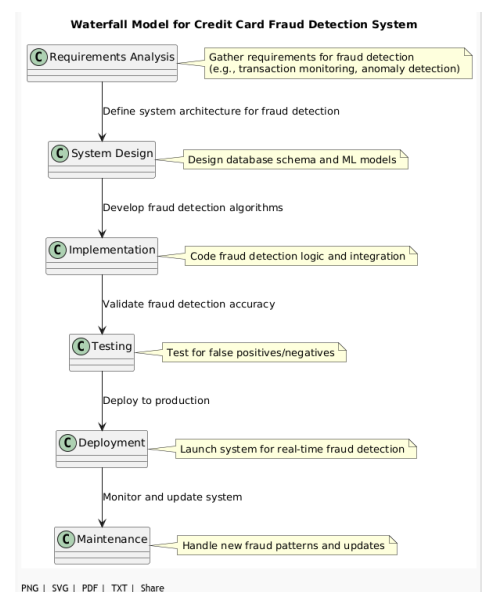
4. Practical Work

4.1 Problem Description

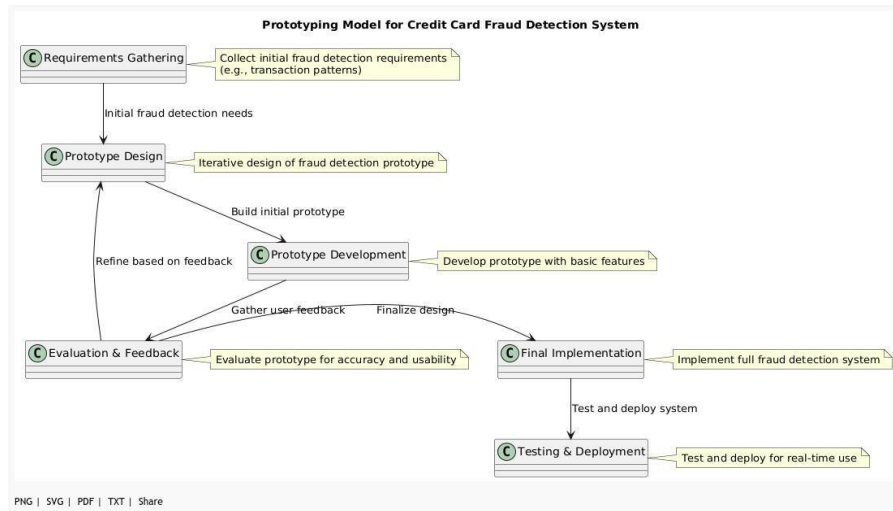
A system monitoring real-time transactions to flag suspicious activity using rules and ML.

4.2 Modeling the System

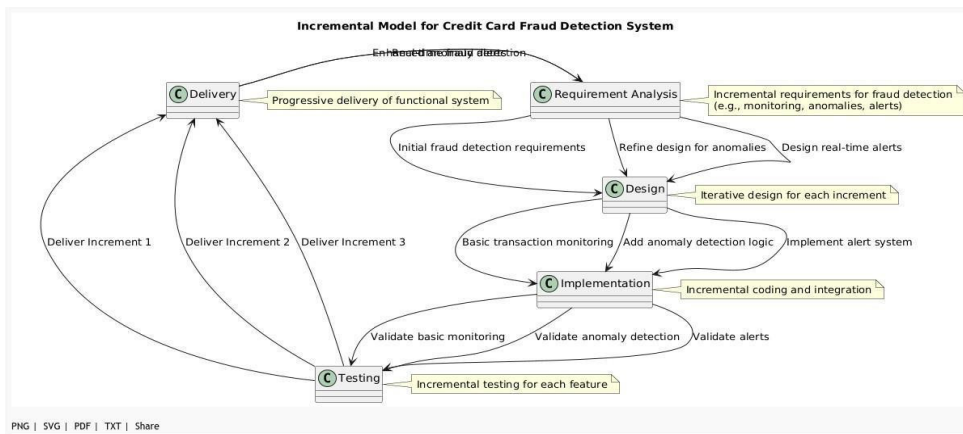
Waterfall: Gathers all fraud patterns initially, proceeding sequentially. High risk, as financial fraud evolves faster than deployment.



Incremental: Excellent fit. Increment 1 builds core logging/rules. Increment 2 adds ML models. Increment 3 enhances dashboards and automated alerts.



Prototyping: Ideal for the analyst UI. A dashboard prototype is built, analysts provide feedback, and it iteratively refines until needs are met.



4.3 Comparison Table

Feature/Model	Waterfall	Incremental	Prototyping
Flexibility	Low	High	Very High
User Involvement	Low	Moderate to High	Very High
Risk Management	Low	Moderate to High	High
Adaptability to ML	Difficult	Easy (add in later increments)	Ideal

5. Conclusion

Model choice depends on the project's nature. Waterfall is too rigid for fraud detection. The Incremental model supports continuous threat updates, and Prototyping clarifies UI requirements. A hybrid approach provides both flexibility and clarity.

Experiment 3

Agile Development Simulation

1. Objective

To understand Agile principles by simulating a Product Backlog, Sprint Backlogs, and a Scrum Board for an Online Food Delivery System.

2. Theory

Agile/Scrum: Iterative approach delivering small, high-value increments in short cycles called Sprints.

Product Backlog: Master list of requirements, ordered by business value.

Sprint Backlog: Subset of items selected for the current Sprint.

Scrum Board: Visual tool tracking Sprint progress (To Do, In Progress, Done).

3. Procedure

Select "Online Food Delivery System".

Create a prioritized Product Backlog with user stories.

Plan two Sprints using high-priority backlog items.

Design a Scrum Board to track tasks.

Conclude on the framework's effectiveness.

4. Practical Work

4.1 Product Backlog

ID	User Story	Priority
PBI-01	I want to sign up and log in.	Must Have
PBI-02	I want to search restaurants.	Must Have
PBI-03	I want to view menus with prices.	Must Have
PBI-04	I want to manage cart items.	Must Have
PBI-05	I want to enter payment info.	Must Have
PBI-06	I want to see delivery times.	Should Have
PBI-07	I want to review restaurants.	Should Have

4.2 Scrum Board

To Do	In Progress	Done
See estimated delivery times Rate and review restaurants	Manage cart items Enter payment info securely	User Sign Up/Login Search restaurants View menus with prices

5. Conclusion

This simulation highlighted Agile's flexibility. Managing Sprints and tracking via a Scrum Board enhanced transparency, making it well-suited for dynamic platforms requiring rapid user feedback.

Experiment 4

Requirements Analysis

1. Objective

To practice requirement identification by developing an SRS for a Student Attendance System, detailing functional/non-functional requirements and Use Cases.

2. Theory

SRS: A comprehensive document outlining system behavior and performance expectations.

Functional Requirements (FR): Define specific interactions (e.g., login, mark attendance).

Non-Functional Requirements (NFR): Define quality attributes (e.g., performance, security).

Use Case Diagram: High-level view of system functionality via actor interactions.

3. Procedure

Select "Student Attendance System".

Identify user actors (Student, Teacher, Admin).

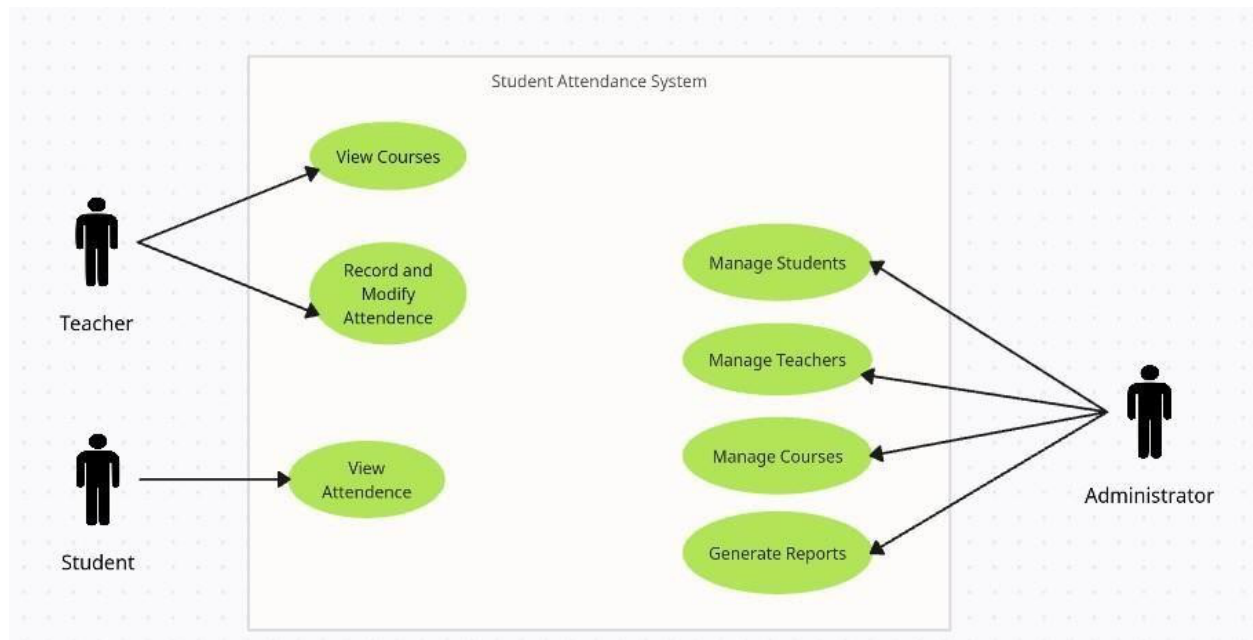
Compile FRs based on user actions.

Compile NFRs for quality attributes.

Organize into an SRS structure and model Use Cases.

4. Practical Work: SRS

ID	Requirement	Details
Functional Requirements		
FR-1	Authentication	Role-based login (Admin, Teacher, Student)
FR-2	Admin Functions	Manage users, courses, and system reports
FR-3	Teacher Functions	Record/modify attendance, view courses
FR-4	Student Functions	View attendance records and course summaries
Non-Functional Requirements		
NFR-1	Performance	Sub 3-second page loads; supports 100+ concurrents
NFR-2	Security	Encrypted passwords, SQLi/XSS protection
NFR-3	Reliability	99.5% uptime with daily backups



5. Conclusion

Proper requirements analysis and SRS documentation act as a project blueprint, preventing scope creep and ensuring stakeholder alignment through clearly defined functional actions and quality expectations.

Experiment 5

UML System Modeling

1. Objective

To practice UML modeling for an Online Banking System by creating Context, Use Case, Sequence, and Activity diagrams.

2. Theory

Context Diagram: High-level system boundaries and external entity interactions.

Use Case Diagram: User functionality perspective mapping actors to actions.

Sequence Diagram: Chronological order of object message exchanges.

Activity Diagram: Control/data flow workflows between activities.

3. Procedure

Select "Online Banking System".

Identify external entities for a Context Diagram.

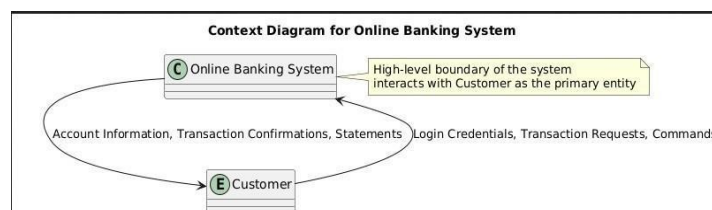
Define main actors/interactions for a Use Case Diagram.

Detail "Transfer Funds" via Sequence Diagram.

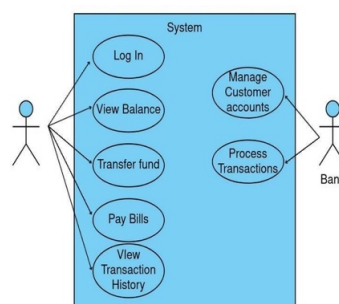
Model "Withdraw Cash" via Activity Diagram.

4. Practical Work

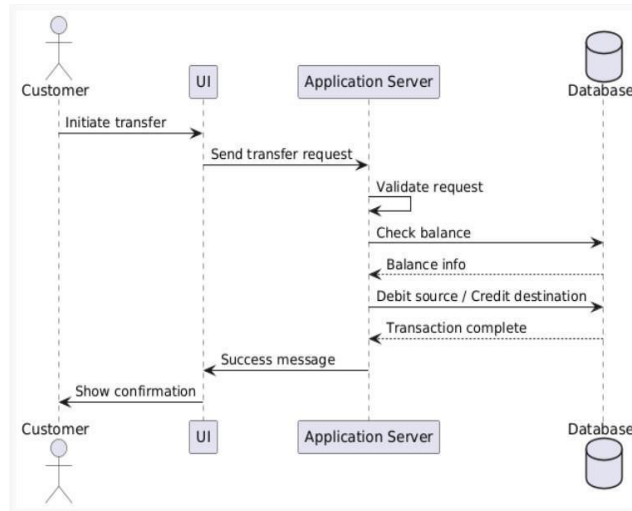
Context Diagram: The Customer sends credentials/requests to the Bank System and receives confirmations.



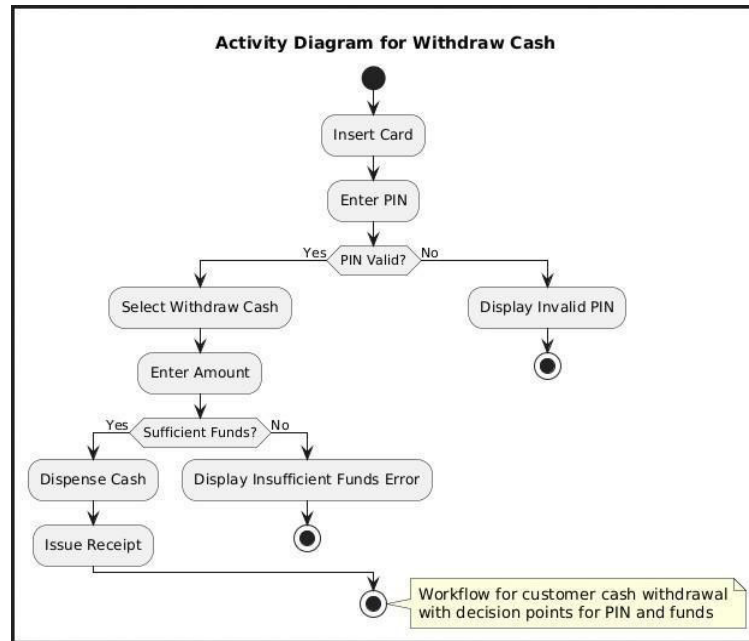
Use Case Diagram: Customers perform Log In, View Balance, Transfer Funds. The internal Bank System manages accounts and processes transactions.



Sequence (Transfer Funds): Customer UI → Server Request → Validate/Check DB Balance → Debit/Credit DB → Success Message to UI.



Activity (Withdraw Cash): Insert Card → Enter PIN → Decision: PIN Valid? → Enter Amount → Decision: Sufficient Funds? → Dispense Cash → End.



5. Conclusion

UML provides a multi-faceted view of software. Utilizing varied diagrams aids in comprehensively documenting system behavior, workflows, and actor interactions for all stakeholders.

Experiment 6

Architectural Design

1. Objective

To explore software architectural patterns by designing the structure of an E-commerce Website.

2. Theory

Layered Architecture: Separates concerns into UI, Business Logic, Data Access, and Data layers.

Client-Server: Splits system into requestors (browsers) and processors (servers).

MVC Pattern: Divides application into Model (data), View (UI), and Controller (input handling).

3. Procedure

Select "E-commerce Website".

Conceptualize structure using Layered Architecture.

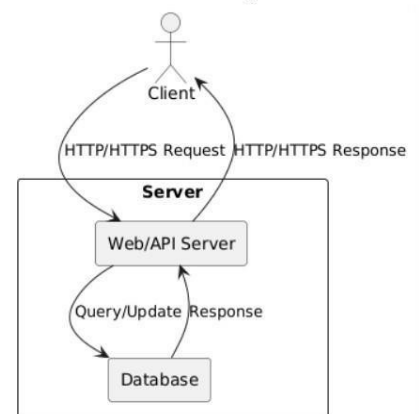
Model interaction via Client-Server logic.

Apply the MVC design pattern for code structure.

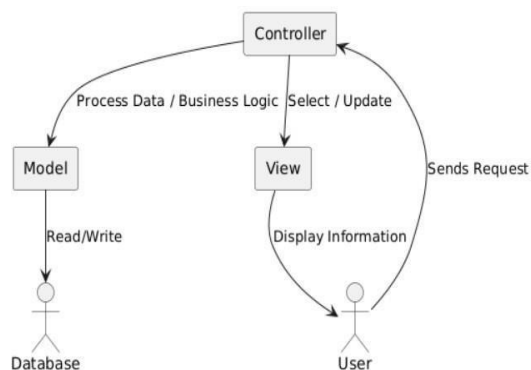
4. Practical Work

Layered Flow: Presentation (Front End) → Business Layer (Logic)
→ Data Access (Bridge) → Data Layer (DB).

Client-Server Flow: The client (browser) sends HTTPS requests. The web/API server processes them, queries the DB, and returns the response.



MVC Structure: The Controller receives user requests, interacts with the Model (Product/Order logic) for DB read/writes, and updates the View displayed to the User.



5. Conclusion

Applying structural patterns enhances maintainability and scalability. Layered and Client-Server models organize macro-interactions, while MVC cleanly separates internal application logic.

Experiment 7

Object-Oriented Design using UML

1. Objective

To apply Object-Oriented Design (OOD) principles to a Hospital Management System.

2. Theory

OOD: System design via objects, emphasizing encapsulation, inheritance, and modularity.

Class Diagram: Blueprint showing class structure, attributes, methods, and associations.

Object Diagram: A real-time snapshot of instantiated objects.

3. Procedure

Identify key classes (Patient, Doctor, Hospital, Registration).

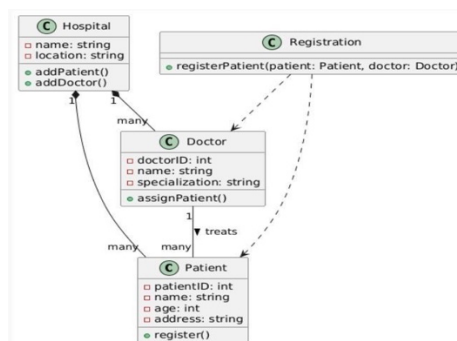
Define attributes and methods.

Establish relationships (Association, Composition).

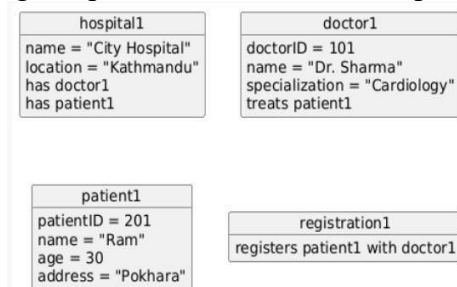
Implement a "Patient Registration" module in Python.

4. Practical Work

Class Diagram: The Class Diagram models the main entities of the hospital. The Patient class holds personal details, while the Doctor class holds information about medical staff.



Object Diagram: The Object Diagram provides a concrete example.



Sample Module Implementation:

The "Patient Registration" module is implemented in Python.

```

class Patient:
    def __init__(self, p_id, name, age, address):
        self.p_id, self.name, self.age, self.address = p_id, name, age, address
    def __str__(self):
        return f"Patient[ID={self.p_id}, Name={self.name}]"

class Doctor:
    def __init__(self, d_id, name, spec):
        self.d_id, self.name, self.spec = d_id, name, spec
    def __str__(self):
        return f"Doctor[ID={self.d_id}, Name={self.name}, Spec={self.spec}]"

class Registration:
    def register(self, patient, doctor):
        return f"{patient.name} assigned to {doctor.name}"

if __name__ == "__main__":
    doc = Doctor(101, "Dr. Sagar", "Neurology")
    pat = Patient(201, "Sagar Genious", 22, "Pokhara")
    reg = Registration()
    print(reg.register(pat, doc))

```

5. Conclusion

OOD provides a logical structure reflecting real-world entities. Class modeling establishes clear component boundaries, simplifying programmatic implementation.

Experiment 8

Testing Techniques

1. Objective

To understand and apply foundational Unit, Black Box, and White Box testing techniques.

2. Theory

Unit Testing: Code-level verification of individual functions.

Black Box Testing: Verifies I/O functionally against requirements, ignoring internal logic.

White Box Testing: Tests internal logic, code structure, and execution paths.

3. Procedure

Write a simple calculator (add/subtract) in Python.

Develop unit tests using a testing framework.

Design Black Box cases based on expected outputs.

Design White Box paths verifying logical branches.

4. Practical Work

```
def add(x, y): return x + y
def subtract(x, y): return x - y
```

```
# Unit Tests Snippet
import unittest
class TestCalc(unittest.TestCase):
    def test_add(self): self.assertEqual(add(15, 5), 20)
    def test_subtract(self): self.assertEqual(subtract(10, 5), 5)
```

Test Cases (Black Box): Input(15,5) for add → Expected: 20. Input(-1,1) for subtract → Expected: -2.

White Box Paths: Testing subtract(a,b) paths: a>b (positive), a<b (negative), a=b (zero result).

5. Conclusion

Comprehensive testing requires all techniques. Unit tests ensure code baseline quality, Black Box validates user expectations, and White Box guarantees logical path integrity.

Experiment 9

Evolution and Maintenance

1. Objective

To execute software evolution by integrating an authentication feature into an existing module and documenting the changes.

2. Theory

Software Evolution updates post-delivery software via:

Corrective: Fixing defects.

Adaptive: Adapting to new environments.

Perfective: Adding new features/functionality.

Preventive: Enhancing maintainability.

3. Procedure

Retrieve Lab 7 Patient Module.

Add username/password attributes and a login method.

Test authentication logic.

Document the maintenance type classification.

4. Practical Work

```
class Patient:
    def __init__(self, p_id, name, user, pwd):
        self.p_id, self.name, self.user, self.pwd = p_id, name, user, pwd
    def login(self, user, pwd):
        if self.user == user and self.pwd == pwd:
            print("Logged in!")
            return True
        return False
```

Documentation: Adding attributes and login method constitutes Perfective maintenance. Testing the integration falls under Corrective validation.

5. Conclusion

Software is dynamic. Evolving an existing module demonstrates the necessity of careful integration planning to avoid breaking current functionality, and accurate documentation ensures future maintainability.

Experiment 10

Project Management Exercise

1. Objective

To apply project management tools (WBS, COCOMO, Gantt) for planning an Online Shopping System.

2. Theory

Work Breakdown Structure (WBS)

A Work Breakdown Structure is a key project deliverable that organizes the team's work into manageable sections. It is a hierarchical decomposition of the total scope of work to be carried out by the project team to accomplish the project objectives and create the required deliverables.

COCOMO Model

The Constructive Cost Model (COCOMO) is a procedural software cost estimation model. The Basic COCOMO model computes software development effort (and cost) as a function of program size, expressed in estimated lines of code (LOC).

The formulas for the Basic COCOMO model are:

- Effort (E) = $a * (KLOC)^b$ (in Person-Months)
- Development Time (D) = $c * (Effort)^d$ (in Months)
- People Required (P) = Effort / Development Time The constants (a, b, c, d) depend on the project type:

Project Type	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Gantt Chart

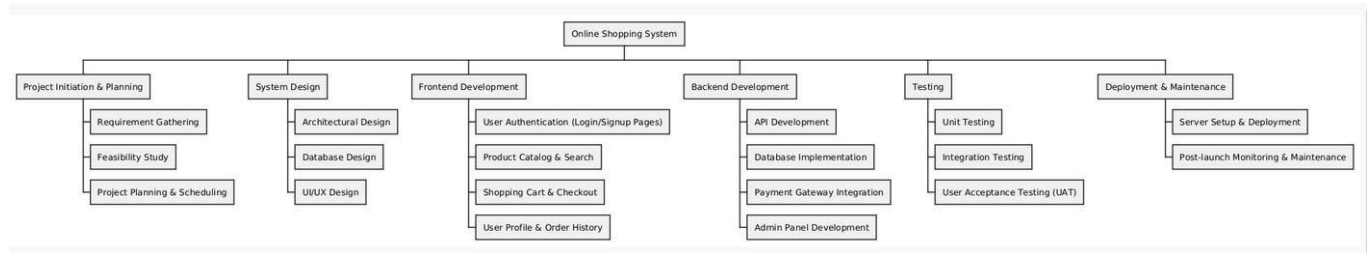
A Gantt chart is a type of bar chart that illustrates a project schedule. It is a common project management tool that shows the start and finish dates of the terminal elements and summary elements of a project.

3. Procedure

1. Draft a WBS for the Online Shopping project phases.
2. Estimate project size at 60 KLOC.
3. Calculate effort and time using Semi-detached COCOMO formulas.
4. Create a Gantt chart representation.

4. Practical Work

Work Breakdown Structure (WBS) for Online Shopping System



COCOMO Model Cost Estimation

Assumptions:

- Project Size: Estimated to be 60 KLOC (60,000 Lines of Code).
- Project Type: Semi-detached (a moderately complex project with a mix of experienced and inexperienced team members).
- Constants for Semi-detached: $a = 3.0$, $b = 1.12$, $c = 2.5$, $d = 0.35$.

Calculations:

Effort (E):

$$E = 3.0 * (60)^{1.12}$$

$$E \approx 265.44 \text{ Person-Months}$$

Development Time (D):

$$D = 2.5 * (265.44)^{0.35}$$

$$D \approx 16.20 \text{ Months}$$

People Required (P):

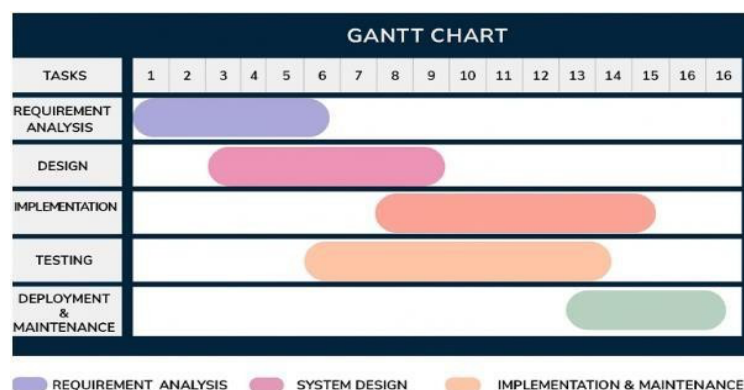
$$P = 265.44 / 16.20$$

$$P \approx 16.4 \text{ People}$$

Estimation Summary:

- Effort: 265 Person-Months
- Schedule: 16 Months
- Average Staffing: 16people

Gantt Chart for Project Scheduling



5. Conclusion

Project management tools are critical for structured execution. WBS organizes tasks, COCOMO provides mathematical resource predictions, and Gantt charts map visual dependencies, establishing a robust framework for timely delivery.

